

CinemaSeat Gateway Service

Payment and OTP provider for the Ultimate Hackathon

It is deliberately unreliable. That is the point. Zayan's OTP that never arrived and his payment that died halfway are not story details, they are this service.

Every team integrates with this same container, so every team faces identical conditions. **Do not write your own mock.**

Run it

```
# docker-compose.yml
services:
  gateway:
    image: asifmahmoud414/mock-gateway:latest
    ports: ["9000:9000"]
```

```
docker compose up gateway
curl localhost:9000/health
```

BEFORE EVENT DAY

Pull the image before you arrive. Venue wifi with 25 teams pulling at once is not where you want to discover a problem.

To build from source instead:

```
docker build -t mock-gateway .
docker run -p 9000:9000 mock-gateway
```

The one mistake everybody makes

Your `callback_url` must be reachable **from inside the gateway container**.

<code>http://localhost:3000/webhooks/payment</code>	WRONG. That is the gateway itself.
<code>http://api:3000/webhooks/payment</code>	RIGHT. Your compose service name.

If callbacks never arrive, check this first, then check `GET /debug/deliveries`.

Endpoints

POST /charge

```
{ "amount": 450, "currency": "BDT",  
  "booking_ref": "bk_001",  
  "callback_url": "http://api:3000/webhooks/payment" }  
  
202 { "payment_id": "pay_abc123", "status": "PENDING" }
```

Returns immediately. The real outcome arrives later, at your `callback_url`.

Optional header **Idempotency-Key**. Send the same key twice and you get the same `payment_id` back without a second charge. Real gateways work this way. Teams that use it are protected against double charging on retry. Teams that do not, are not.

POST /refund

```
{ "payment_id": "pay_abc123" } -> 202 { "status": "PENDING" }
```

404 if unknown, 409 if the payment did not succeed. A REFUNDED callback follows.

POST /otp/send

```
{ "phone": "01700000000", "ref": "bk_001",  
  "callback_url": "http://api:3000/webhooks/otp" }
```

202. The code is delivered to `callback_url` after a delay, if it is delivered at all. About 10% are silently lost, exactly like Zayan's.

POST /otp/verify

```
{ "ref": "bk_001", "code": "123456" } -> 200 { "verified": true }
```

400 on wrong or expired code, 429 after 5 attempts.

GET /health

Always fast. Use it for your compose healthcheck.

The callback

The gateway POSTs this to your `callback_url`:

```
{ "event_id": "evt_9f2a...",  
  "payment_id": "pay_abc123",  
  "booking_ref": "bk_001",  
  "status": "SUCCEEDED",
```

```
"amount": 450,  
"currency": "BDT",  
"timestamp": "2026-08-08T11:03:22.418Z" }
```

status is SUCCEEDED, FAILED or REFUNDED.

Headers: X-Signature (HMAC-SHA256 of the raw body, see below) and X-Gateway-Event.

Three rules

- 1. Always return 2xx, even for a duplicate.** Anything else is read as a delivery failure and you will be retried with exponential backoff, up to 8 times. Return 200 and handle the duplicate quietly.
- 2. event_id is your deduplication key.** A duplicate delivery carries the same event_id as the original. If you have seen it, ignore it.
- 3. Callbacks can arrive before you are ready.** Occasionally the callback lands before your /pay handler has even finished writing the payment_id. Design for it.

Documented misbehaviour

Not a bug list. A specification.

BEHAVIOUR	DEFAULT RATE
Callback delayed 2 to 15 seconds	always
Payment fails (FAILED)	10%
Same callback delivered twice, same event_id	8%
/charge returns 500	2%
OTP never delivered	10%
Callback arrives before /charge responds	force header only
Retries your callback on non-2xx	up to 8 times, exponential backoff

Control headers

Send these on /charge while building and testing.

HEADER	EFFECT
X-Mock-Mode: deterministic	2 second delay, always succeeds, never duplicates
X-Mock-Force: success	Guaranteed clean success
X-Mock-Force: fail	Guaranteed FAILED
X-Mock-Force: duplicate	Guaranteed duplicate callback
X-Mock-Force: timeout	Hangs 30s, then drops the connection
X-Mock-Force: race	Callback lands before /charge responds

X-Mock-Mode: deterministic also works on /refund and /otp/send. In deterministic mode the OTP code is always **123456**.

READ THIS TWICE

Judges use the force headers. Every team is tested on identical conditions rather than on luck. Make sure each of these behaves correctly before you freeze.

Deterministic mode is for building. Nothing interesting happens with it on. Turn it off before you believe your own results.

Debug endpoints

Not part of a real gateway. Provided so you can see what was done to you.

ENDPOINT	WHAT IT GIVES YOU
GET /debug/deliveries	Every callback attempt, with HTTP status and errors. Start here when callbacks seem missing.
GET /debug/deliveries?booking_ref=bk_001	Filtered to one booking
GET /debug/payments	All payments and their state
GET /debug/payments/:id	One payment
GET /debug/otp/:ref	The OTP record, including the code
GET /debug/config	Active configuration
POST /debug/reset	Wipe all state

Configuration

Defaults are what ships on event day. Override only for local testing.

VARIABLE	DEFAULT
PORT	9000
MIN_DELAY_MS / MAX_DELAY_MS	2000 / 15000
FAILURE_RATE	0.10
DUPLICATE_RATE	0.08
CHARGE_ERROR_RATE	0.02
RACE_RATE	0 (force header only)
OTP_FAILURE_RATE	0.10
DUPLICATE_GAP_MS	3000
MAX_CALLBACK_ATTEMPTS	8
CALLBACK_TIMEOUT_MS	5000
GATEWAY_SECRET	z2p-2026-secret

State is in memory. Restarting the container wipes everything.

Verifying the signature (bonus marks)

Anyone on the internet can POST to your webhook path. Verify the HMAC.

```
const expected = crypto
  .createHmac('sha256', process.env.GATEWAY_SECRET)
  .update(rawBody)           // the raw body, before JSON parsing
  .digest('hex');

if (req.get('X-Signature') !== expected) return res.sendStatus(401);
```

Compute it over the raw request body. If your framework parses and re-serialises the JSON first, the bytes change and the signature will never match.

Quick smoke test

```
curl -s -X POST localhost:9000/charge \  
-H 'Content-Type: application/json' \  
-H 'X-Mock-Mode: deterministic' \  
-d '{"amount":450,"currency":"BDT","booking_ref":"bk_test",  
    "callback_url":"http://api:3000/webhooks/payment"}'  
  
sleep 4  
curl -s "localhost:9000/debug/deliveries?booking_ref=bk_test" | jq
```

If ok: false shows up there, the gateway reached out and your app did not answer properly. The http_status and error fields tell you which.

LEARN IT > BUILD IT > SHIP IT

IEEE Computer Society, CUET Student Branch Chapter · In partnership with Poridhi.io